

Lab 2: Pipes, Functions, and Conditionals

Use a pipe, then write a function with a conditional and a check on the inputs.

Work in a **Quarto** document (or R Markdown if you already started one). Number the exercises. Keep only the code that answers the question.

Do **2.1** through **2.3** first. You will need a pipe, `function()`, `if / else`, and `stopifnot()`.

1. Using a pipe

Use a pipe to:

1. draw 100 values from a normal distribution with mean 0 and standard deviation 8
2. take absolute values
3. take square roots
4. replace each value by the largest integer that is still smaller than it
5. show the first 10 results

Do this **without** intermediate objects.

Useful functions: `rnorm()`, `abs()`, `sqrt()`, `floor()`, `head()`.

2. Write `add_half()`

Write a function of two arguments: a numeric vector `x` and a numeric scalar `y`.

The returned vector should keep the first half of `x` unchanged, and add `y` to the second half.

If `length(x)` is odd, the middle value is a random choice: either the middle of `x`, or the middle of `x + y`.

Choose a clear name if you prefer another one, but the examples below use `add_half()`. Write a short documentation header.

Useful functions: `length()`, `%%` (even vs odd), `floor()`, `sample()`, `c()`, and `[...]`. You will need a conditional.

Example when the length is odd. The middle `3` happened to be kept; another run could have used `4`:

```
x <- c(1, 1, 3, 1, 1)
y <- 1
add_half(x, y)
# [1] 1 1 3 2 2
```

Example when the length is even:

```
x <- c(1, 1, 2, 2)
y <- 1
add_half(x, y)
# [1] 1 1 3 3
```

Plan the steps before you type:

1. take `x` and `y`
 2. find `length(x)`
 3. decide even or odd (`%%`)
 4. if even: take half the length; build `c(first_half, second_half + y)`
 5. if odd: `floor()` of half the length; `sample()` the middle from `x` or `x + y`; build `c(first_half, middle, second_half + y)`
 6. return the new vector
-

3. Check the inputs

Add `stopifnot()` so `x` is a numeric vector and `y` is a single numeric value. `?length`, `?is.numeric`, and `?is.vector` are enough.

These calls should fail:

```
x <- c(1, 1, 1, 1)
y <- c(1, 2)
add_half(x, y)
# Error in add_half(x, y): length(y) == 1 is not TRUE

x <- c(1, 1, 1, 1)
y <- "0"
add_half(x, y)
# Error in add_half(x, y): is.numeric(y) is not TRUE
```

Show at least two failing checks in your document, including the error.